

limma

Kasper D. Hansen

- [Dependencies](#)
- [Overview](#)
- [Other Resources](#)
- [Analysis Setup and Design](#)
- [A two group comparison](#)
 - [Obtaining data](#)
 - [A linear model](#)
- [More on the design](#)
- [Background: Data representation in limma](#)
- [Background: The targets file](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(limma)
library(leukemiasEset)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("limma", "leukemiasEset"))
```

Overview

limma is a very popular package for analyzing microarray and RNA-seq data.

LIMMA stands for “linear models for microarray data”. Perhaps unsurprisingly, limma contains functionality for fitting a broad class of statistical models called “linear models”. Examples of such models include linear regression and analysis of variance. While most of the functionality of limma has been developed for microarray data, the model fitting routines of limma are useful for many types of data, and is not limited to microarrays. For example, I am using limma in my research on analysis of DNA methylation.

Other Resources

- The limma User's Guide from the [limma webpage](#). This is an outstanding piece of documentation which has (rightly) been called “the best resource on differential expression analysis available”.

Analysis Setup and Design

(The discussion in this section is not specific to limma.)

A very common analysis setup is having access to a matrix of numeric values representing some measurements; an example is gene expression. Traditionally in Bioconductor, and in computational biology more generally, columns of this matrix are samples and rows of the matrix are features. Features can be many things; in gene expression a feature is a gene. The feature by sample layout in Bioconductor is the transpose of the layout in classic statistics where the matrix is samples by features; this sometimes cause confusion.

A very common case of this type of data is gene expression data (either from microarrays or from RNA sequencing) where the features are individual genes.

Samples are usually few (usually less than a hundred, almost always less than a thousand) and are often grouped; arguably the most common setup is having samples from two different groups which we can call cases and controls. The objective of an analysis is frequently to discover which features (genes) are different between groups or stated differently: to discover which genes are differentially expressed between cases and controls. Of course, more complicated designs are also used; sometimes more than two groups are considered and sometimes there are additional important covariates such as age or sex. An example of a more complicated design is a time series experiment where each time point is a group and where it is sometimes important to account for the time elapsed between time points.

Broadly speaking, how samples are distributed between groups determines the **design** of the study. In addition to the design, there is one or more question(s) of interest(s) such as the difference between two groups. Such questions are usually formalized as **contrasts**; an example of a contrast is indeed the difference between two groups.

Samples are usually assumed to be independent but are sometimes paired; an example of pairing is when a normal and a cancer sample from the same patient is available. Pairing allows for more efficient inference because each sample has a sample specific control. If only some samples are paired, or if multiple co-linked samples exists, it becomes harder (but usually possible) to account for this structure in the statistical model.

As stated above, the most common design is a two group design with unpaired samples.

Features are often genes or genomic intervals, for example different promoters or genomic bins. The data is often gene expression data but could be histone modification abundances or even measurements from a Hi-C contact matrix.

Common to all these cases is the rectangular data structure (the matrix) with samples on columns and features on rows. This is exactly the data structure which is represented by an `ExpressionSet` or a `SummarizedExperiment`.

A number of different packages allows us to fit common types of models to this data structure

- *limma* fits a so-called linear model; examples of linear models are (1) linear regression, (2) multiple linear regression and (3) analysis of variance.
- *edgeR*, *DESeq* and *DESeq2* fits generalized linear models, specifically models based on the negative binomial distribution.

Extremely simplified, *limma* is useful for continuous data such as microarray data and *edgeR* / *DESeq* / *DESeq2* are useful for count data such as high-throughput sequencing. But that is a very simplified statement.

In addition to the distributional assumptions, all of these packages uses something called empirical Bayes techniques to borrow information across features. As stated above, usually the number of samples is small and the number of features is large. It has been shown time and time again that you can get better results by borrowing information across features, for example by modeling a mean-variance relationship. This can be done in many ways and often depends on the data characteristics of a specific type of data. For example both *edgeR* and *DESeq2* are very popular in the analysis of RNA-seq data and all three packages uses models based on the negative binomial distribution. For a statistical point of view, a main difference between these packages (models) is how they borrow information across genes.

Fully understanding these classes of models as well as their strengths and limitations are beyond our scope. But we will still introduce aspects of these packages because they are so widely used.

A two group comparison

Obtaining data

Let us use the `leukemiasEset` dataset from the *leukemiasEset* package; this is an `ExpressionSet`.

```
library(leukemiasEset)
data(leukemiasEset)
leukemiasEset
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 20172 features, 60 samples
##   element names: exprs, se.exprs
## protocolData
##   sampleNames: GSM330151.CEL GSM330153.CEL ... GSM331677.CEL (60
##     total)
##   varLabels: ScanDate
##   varMetadata: labelDescription
## phenoData
##   sampleNames: GSM330151.CEL GSM330153.CEL ... GSM331677.CEL (60
##     total)
##   varLabels: Project Tissue ... Subtype (5 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
## Annotation: genemapperhgu133plus2
```

```
table(leukemiasEset$LeukemiaType)
```

```
##
## ALL  AML  CLL  CML  NoL
##   12   12   12   12   12
```

This is data on different types of leukemia. The code NoL means not leukemia, ie. normal controls.

Let us ask which genes are differentially expressed between the ALL type and normal controls. First we subset the data and clean it up

```
ourData <- leukemiasEset[, leukemiasEset$LeukemiaType %in% c("ALL", "NoL")
ourData$LeukemiaType <- factor(ourData$LeukemiaType)
```

A linear model

Now we do a standard limma model fit

```

design <- model.matrix(~ ourData$LeukemiaType)
fit <- lmFit(ourData, design)
fit <- eBayes(fit)
topTable(fit)

```

```

##           logFC AveExpr      t      P.value      adj.P.Val
## ENSG00000163751 4.089587 5.819472 22.51729 9.894742e-18 1.733025e-13
## ENSG00000104043 4.519488 5.762115 21.98550 1.718248e-17 1.733025e-13
## ENSG00000008394 5.267835 7.482490 20.08250 1.374231e-16 9.240332e-13
## ENSG00000165140 3.206807 6.560163 19.41855 2.959391e-16 1.492421e-12
## ENSG00000204103 4.786273 7.774809 19.04041 4.628812e-16 1.867448e-12
## ENSG00000145569 2.845963 5.958707 18.46886 9.239404e-16 3.067090e-12
## ENSG00000038427 5.047670 6.496822 18.35375 1.064328e-15 3.067090e-12
## ENSG00000173391 4.282498 5.293222 17.89645 1.881511e-15 4.744229e-12
## ENSG00000138449 5.295928 6.999716 17.79655 2.134448e-15 4.784010e-12
## ENSG00000105352 2.521351 7.054018 17.62423 2.657074e-15 5.359850e-12
##                                     B
## ENSG00000163751 30.15253
## ENSG00000104043 29.65066
## ENSG00000008394 27.73589
## ENSG00000165140 27.02056
## ENSG00000204103 26.60138
## ENSG00000145569 25.95084
## ENSG00000038427 25.81729
## ENSG00000173391 25.27803
## ENSG00000138449 25.15836
## ENSG00000105352 24.95031

```

What happens here is a common limma (and friends) workflow. First, the comparison of interest (and the design of the experiment) is defined through a so-called “design matrix”. This matrix basically encompasses everything we know about the design; in this case there are two groups (we have more to say on the design below). Next, the model is fitted. This is followed by borrowing strength across genes using a so-called empirical Bayes procedure (this is the step in limma which really works wonders). Because this design only has two groups there is only one possible comparison to make: which genes differs between the two groups. This question is examined by the `topTable()` function which lists the top differentially expressed genes. In a more complicated design, the `topTable()` function would need to be told which comparison of interest to summarize.

An important part of the output is `logFC` which is the log fold-change. To interpret the sign of this quantity you need to know if this is ALL-NoL (in which case positive values are up-regulated in ALL) or the reverse. In this case this is determined by the reference level which is the first level of the factor.

```
ourData$LeukemiaType
```

```
## [1] ALL ALL ALL ALL ALL ALL ALL ALL ALL ALL ALL ALL ALL NoL NoL NoL NoL  
## [18] NoL NoL NoL NoL NoL NoL NoL  
## Levels: ALL NoL
```

we see the reference level is ALL so positive values means it is down-regulated in cancer. You can change the reference level of a factor using the `relevel()` function. You can also confirm this by computing the `logFC` by hand, which is useful to know. Let's compute the fold-change of the top differentially expressed gene:

```
topTable(fit, n = 1)
```

```
##           logFC AveExpr      t      P.value  adj.P.val  
## ENSG00000163751 4.089587 5.819472 22.51729 9.894742e-18 1.733025e-13  
##           B  
## ENSG00000163751 30.15253
```

```
genename <- rownames(topTable(fit, n=1))  
typeMean <- tapply(exprs(ourData)[genename,], ourData$LeukemiaType, mean,  
typeMean
```

```
##      ALL      NoL  
## 3.774678 7.864265
```

```
typeMean["NoL"] - typeMean["ALL"]
```

```
##      NoL  
## 4.089587
```

confirming the statement. It is sometimes useful to check things by hand to make sure you have the right interpretation. Finally, note that limma doesn't do anything different from a difference of means when it computes `logFC`; all the statistical improvements centers on computing better t-statistics and p-values.

The reader who has some experience with statistics will note that all we are doing is comparing two groups; this is the same setup as the classic t-statistic. What we are computing here is indeed a t-statistic, but one where the variance estimation (the denominator of the t-statistics) is *moderated* by borrowing strength across genes (this is what `eBayes()` does); this is called a moderated t-statistic.

The output from `topTable()` includes

- `logFC`: the log fold-change between cases and controls.
- `t`: the t-statistic used to assess differential expression.
- `P.value`: the p-value for differential expression; this value is not adjusted for multiple testing.
- `adj.P.val`: the p-value adjusted for multiple testing. Different adjustment methods are available, the default is Benjamini-Horchberg.

How to setup and interpret a design matrix for more complicated designs is beyond the scope of this course. The limma User's Guide is extremely helpful here. Also, note that setting up a design matrix for an experiment is a standard task in statistics (and requires very little knowledge about genomics), so other sources of help is a local, friendly statistician or text books on basic statistics.

More on the design

In the analysis in the preceding section we setup our model like this

```
design <- model.matrix(~ ourData$LeukemiaType)
```

and then we use F-statistics to get at our question of interest. We can do this easily because there is only really one interesting question for this design: is there differential expression between the two groups. But this formulation did not use contrasts; in the "Analysis Setup and Design" section we discussed how one specifies the question of interest using contrasts and we did not really do this here.

Let's try. A contrast is interpreted relative to the design matrix one uses. One conceptual design may be represented by different design matrices, which is one of the reasons why design matrices and contrasts take a while to absorb.

Let's have a look

```
head(design)
```

```
##      (Intercept) ourData$LeukemiaTypeNoL
## 1              1              0
## 2              1              0
## 3              1              0
## 4              1              0
## 5              1              0
## 6              1              0
```

This matrix has two columns because there are two parameters in this conceptual design: the expression level in each of the two groups. In this **parametrization** column 1 represents the expression of the ALL group and column 2 represents the difference in expression level from the NoL group to the ALL group. Testing that the two groups have the same expression level is done by testing whether the second parameter (equal to the difference in expression between the two groups) is equal to zero.

A different parametrization is

```
design2 <- model.matrix(~ ourData$LeukemiaType - 1)
head(design2)
```

```
##      ourData$LeukemiaTypeALL ourData$LeukemiaTypeNoL
## 1              1              0
## 2              1              0
## 3              1              0
## 4              1              0
## 5              1              0
## 6              1              0
```

```
colnames(design2) <- c("ALL", "NoL")
```

In this design, the two parameters corresponding to the two columns of the design matrix, represents the expression levels in the two groups. And the scientific question gets translated into asking whether or not these two parameters are the same. Let us see how we form a contrast matrix for this


```
fit2 <- lmFit(ourData, design2)
contrast.matrix <- makeContrasts("ALL-NoL", levels = design2)
contrast.matrix
```

```
##           Contrasts
## Levels ALL-NoL
##      ALL      1
##      NoL     -1
```

Here we say we are interested in ALL-NoL which has the opposite sign of what we were doing above (where it was NoL-ALL; since NoL is the natural reference group this makes a lot more sense. Now we fit

```
fit2C <- contrasts.fit(fit2, contrast.matrix)
fit2C <- eBayes(fit2C)
topTable(fit2C)
```

```
##           logFC AveExpr      t      P.value      adj.P.Val
## ENSG00000163751 -4.089587  5.819472 -22.51729 9.894742e-18 1.733025e-1
## ENSG00000104043 -4.519488  5.762115 -21.98550 1.718248e-17 1.733025e-1
## ENSG00000008394 -5.267835  7.482490 -20.08250 1.374231e-16 9.240332e-1
## ENSG00000165140 -3.206807  6.560163 -19.41855 2.959391e-16 1.492421e-1
## ENSG00000204103 -4.786273  7.774809 -19.04041 4.628812e-16 1.867448e-1
## ENSG00000145569 -2.845963  5.958707 -18.46886 9.239404e-16 3.067090e-1
## ENSG00000038427 -5.047670  6.496822 -18.35375 1.064328e-15 3.067090e-1
## ENSG00000173391 -4.282498  5.293222 -17.89645 1.881511e-15 4.744229e-1
## ENSG00000138449 -5.295928  6.999716 -17.79655 2.134448e-15 4.784010e-1
## ENSG00000105352 -2.521351  7.054018 -17.62423 2.657074e-15 5.359850e-1
##                               B
## ENSG00000163751 30.15253
## ENSG00000104043 29.65066
## ENSG00000008394 27.73589
## ENSG00000165140 27.02056
## ENSG00000204103 26.60138
## ENSG00000145569 25.95084
## ENSG00000038427 25.81729
## ENSG00000173391 25.27803
## ENSG00000138449 25.15836
## ENSG00000105352 24.95031
```

Note that this is exactly the same output from `topTable()` as above, except for the sign of the `LogFC` column.

Background: Data representation in limma

As we see above, *limma* works directly on `ExpressionSets`. It also works directly on matrices. But *limma* also have a class `RGList` which represents a two-color microarray. The basic data stored in this class is very `ExpressionSet`-like, but it has at least two matrices of expression measurements `R` (Red) and `G` (Green) and optionally two additional matrices of background estimates (`Rb` and `Gb`). It has a slot called `genes` which is basically equivalent to `featureData` for `ExpressionSets` (ie. information about which genes are measured on the microarray) as well as a `targets` slot which is basically the `pData` information from `ExpressionSet`.

Background: The targets file

limma introduced the concept of a so-called `targets` file. This is just a simple text file (usually `TAB` or comma-separated) which holds the phenotype data. The idea is that it is easier for many users to create this text file in a spreadsheet program, and then read it into `R` and stored the information in the data object.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] parallel  methods    stats      graphics  grDevices  utils      datas
## [8] base
##
## other attached packages:
## [1] leukemiasEset_1.4.0 Biobase_2.28.0      BiocGenerics_0.14.0
## [4] limma_3.24.15      BiocStyle_1.6.0     rmarkdown_0.8
##
## loaded via a namespace (and not attached):
## [1] magrittr_1.5      formatR_1.2        tools_3.2.1        htmltools_0.2.6
## [5] yaml_2.1.13       stringi_0.5-5      knitr_1.11         stringr_1.0.0
## [9] digest_0.6.8      evaluate_0.7.2
```

